Write-Ahead Logging

Background: Disk write buffer

Normally, disk writes go to a buffer in memory, instead of physically writing to disk.

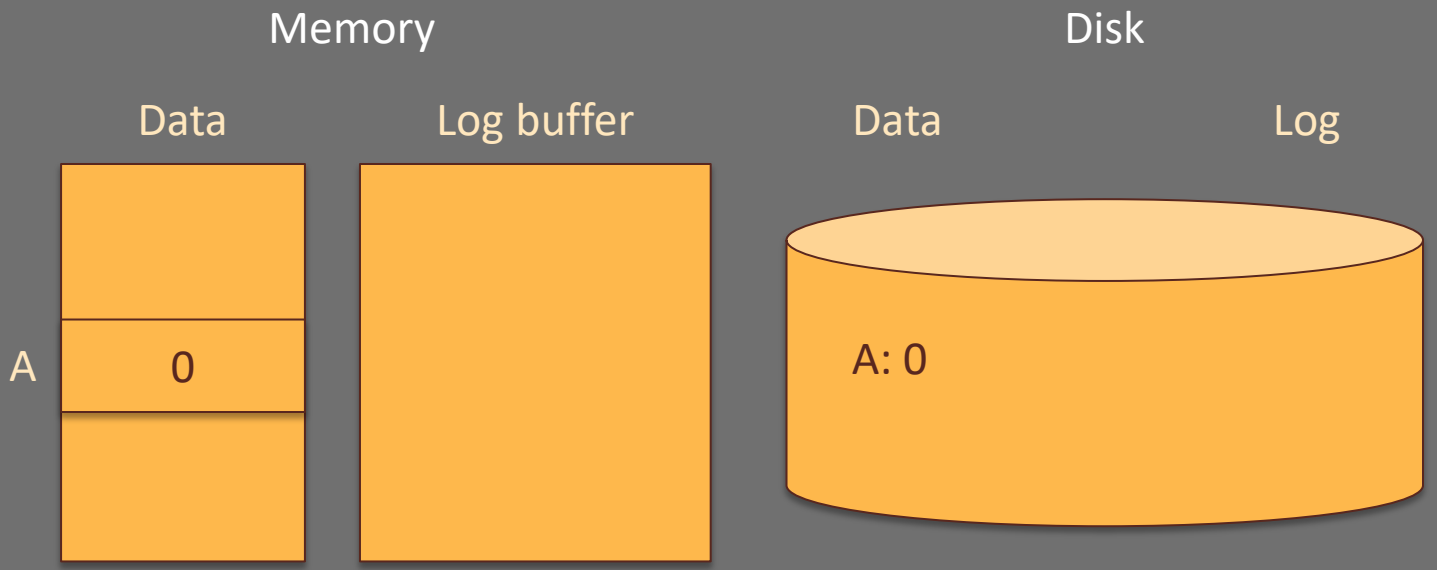
Buffer written to disk:

- When full
- When flushed
- On disk seek – i.e., writes are non-sequential

Write-ahead logging

Transaction T

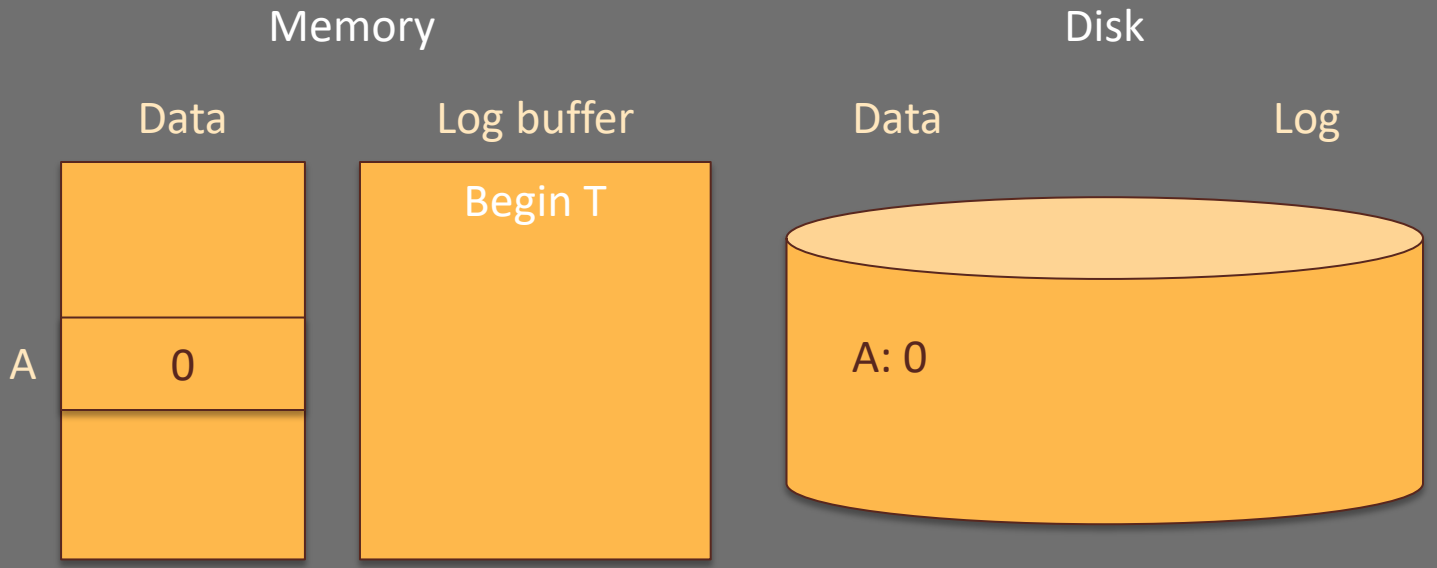
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

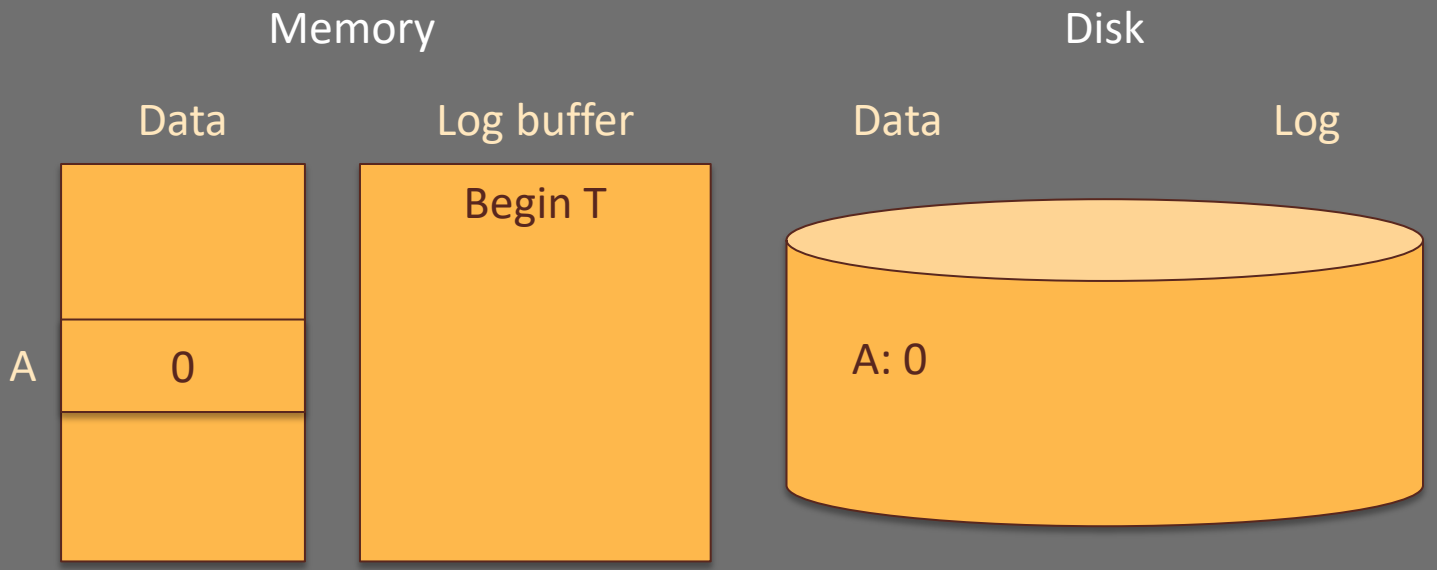
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

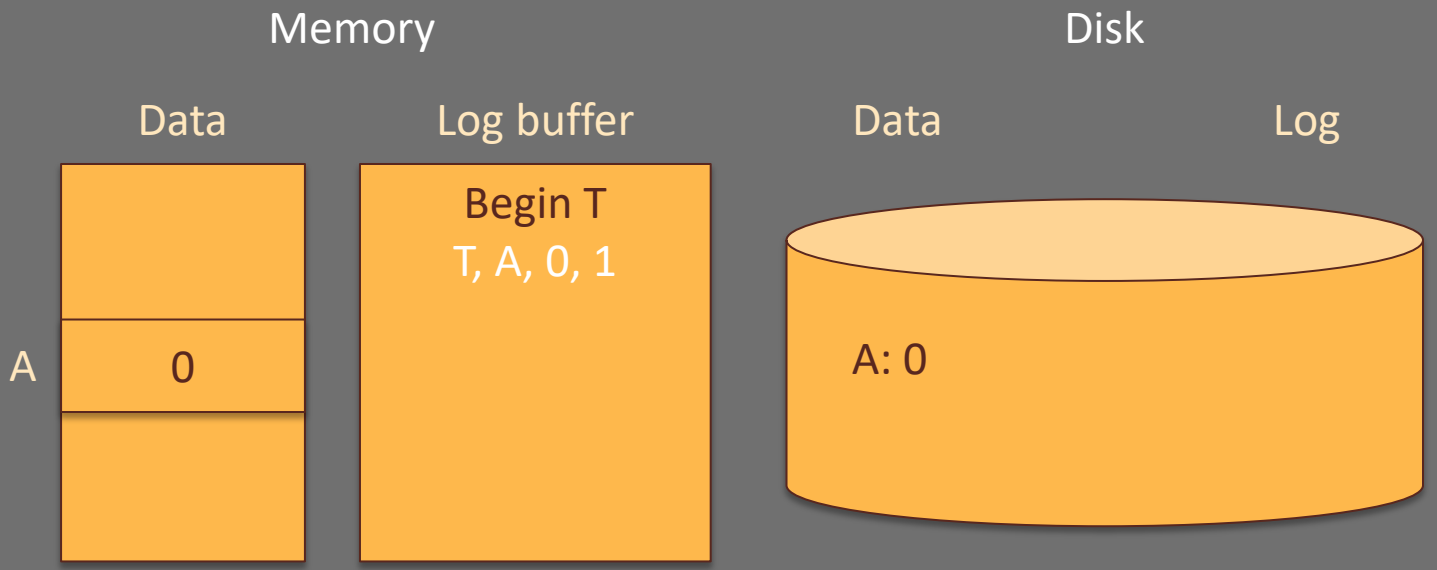
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

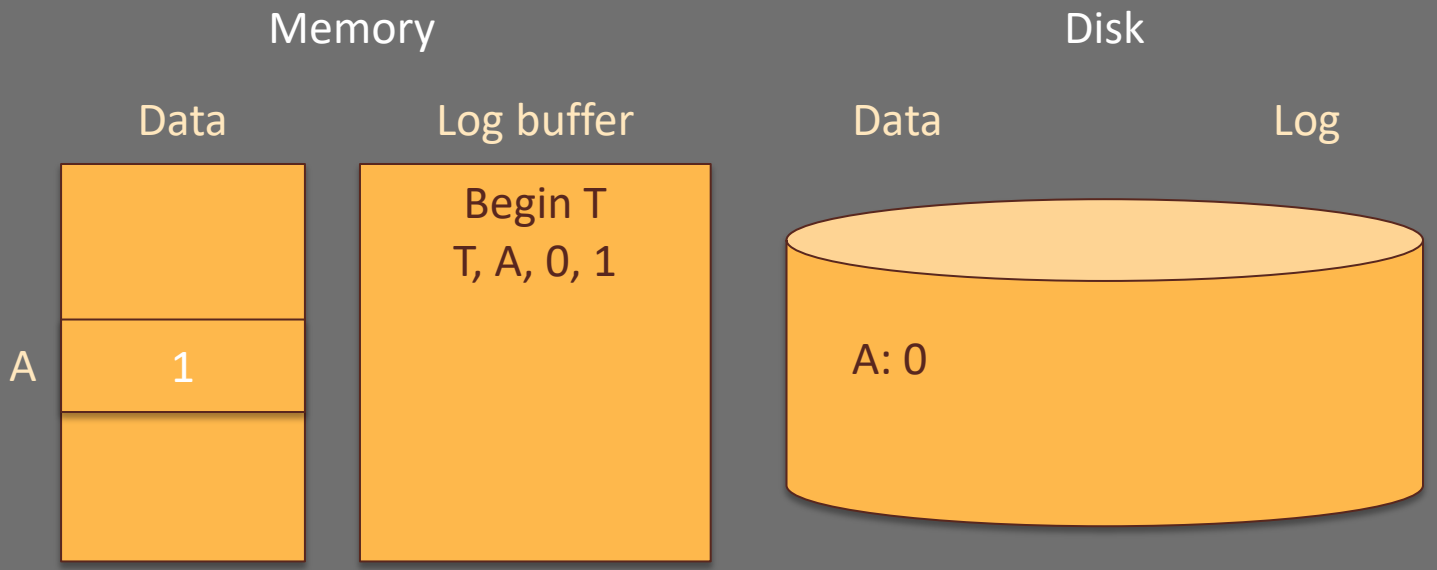
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

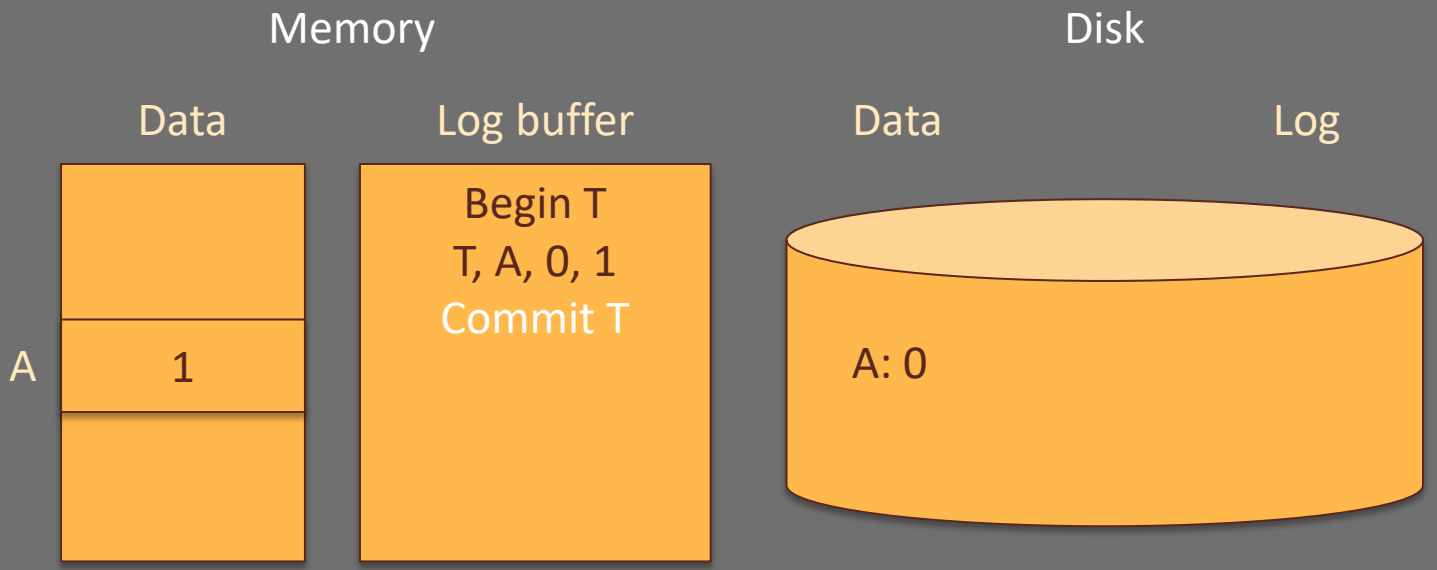
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

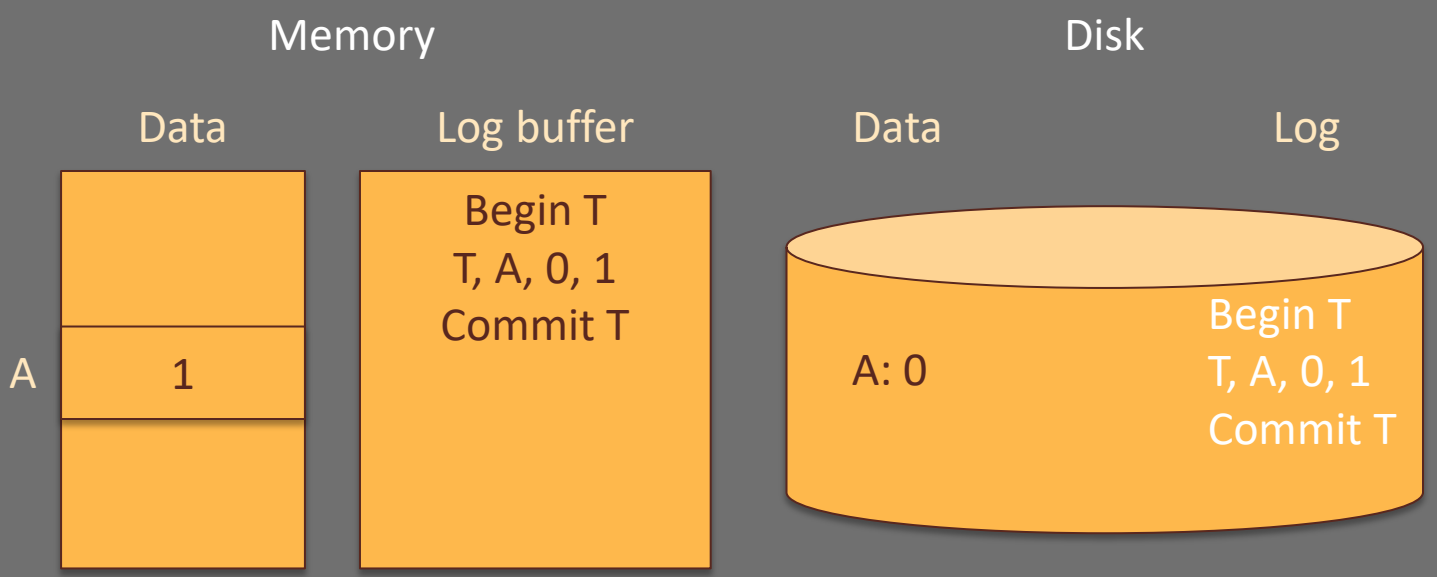
R(A)
W(A = A+1)
Commit



Write-ahead logging

Transaction T

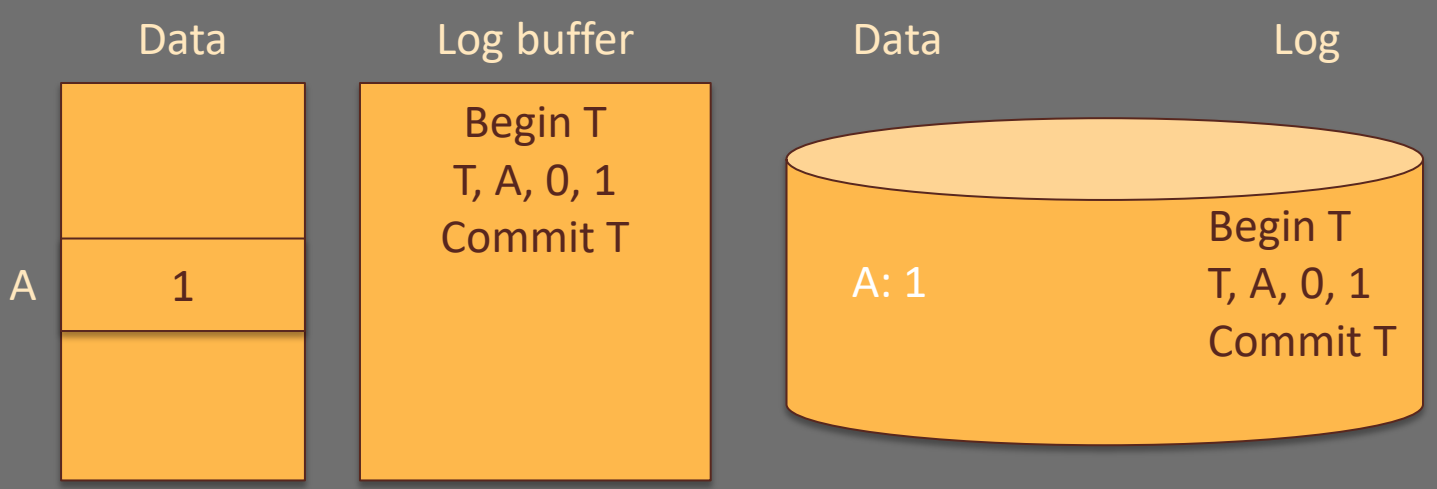
R(A)
W(A = A+1)
Commit
1. Log



Write-ahead logging

Transaction T

R(A)
W(A = A+1)
Commit
1. Log
2. Data





One more detail

If a memory page needs to be written during transaction, first flush log.

Use log to undo, if transaction aborted.

PostgreSQL transaction logging

- Problem with write-ahead logging:
  - Commits (and disk writes) can be frequent.
  - Disk writes are expensive.

- Can we reduce disk writes during logging?

When to write to disk?

- **Not** at commit.
- **Only** at checkpoint.
  - Also discard log at checkpoint, since disk & memory are consistent.
  - Log snapshot not written to disk, so any disk restore is via RAID or a backup.

Atomicity – Writes only completed on commit.

Durability – Once committed, data is in stable storage (at least in log).